CST-407 Activity 5 Spring Boot Security Configuration

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

08/19/2025

Table of Contents

Part 1 — Adding Security to a Spring Boot Application

1.1 Starting Project

Database Config Updated for MAMP

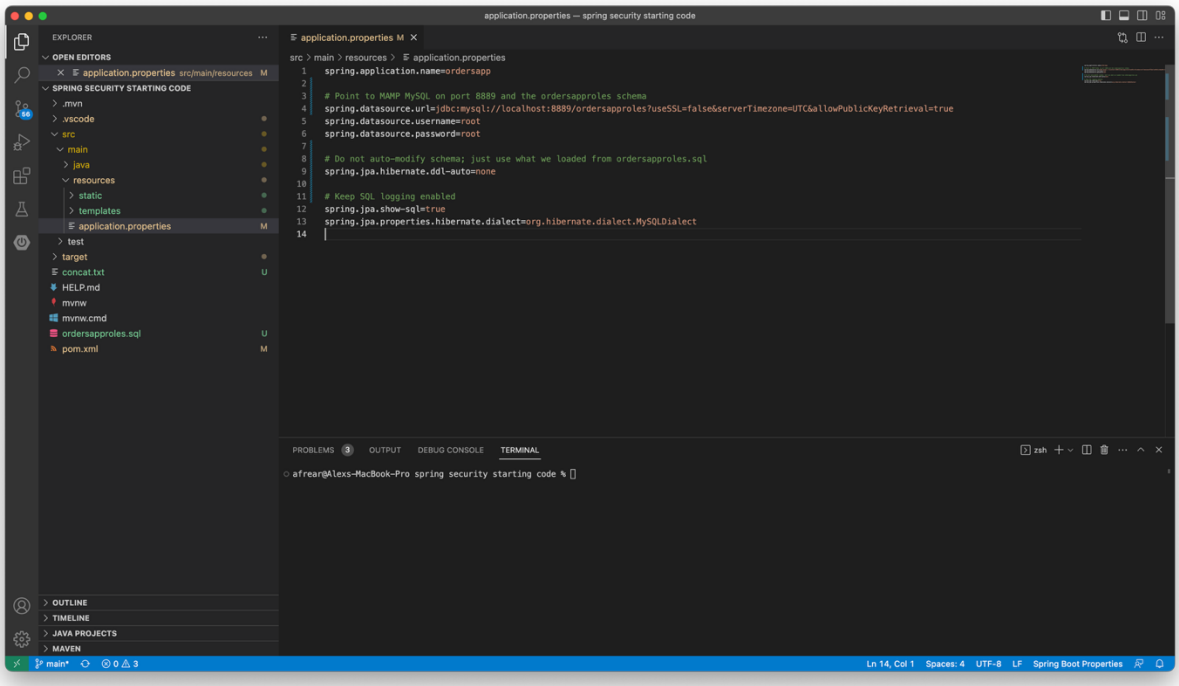

Figure 1 `application.properties` updated to use port 8889 and `ddl-auto=none`, matching the `ordersapproles` schema in MySQL Workbench.

Unsecured Orders Page (Before Security)

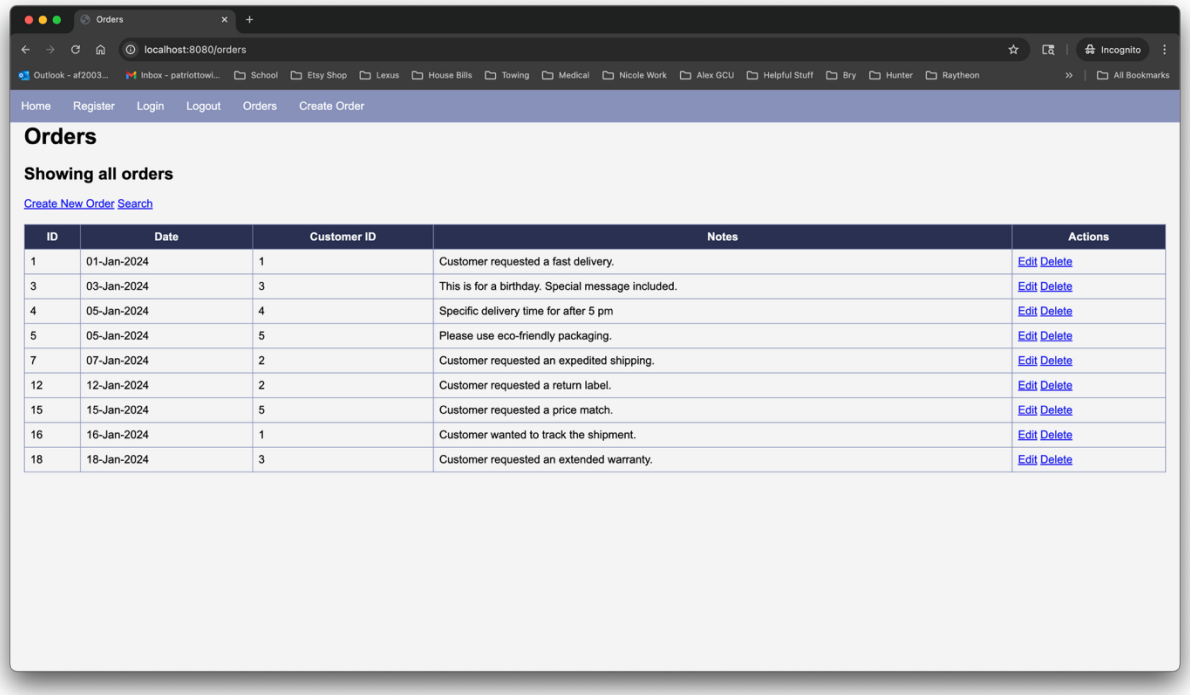

Figure 2 Orders page allows CRUD with no login, confirming security hasn't been applied yet.

1.2 Add Dependencies

Security Dependency Added

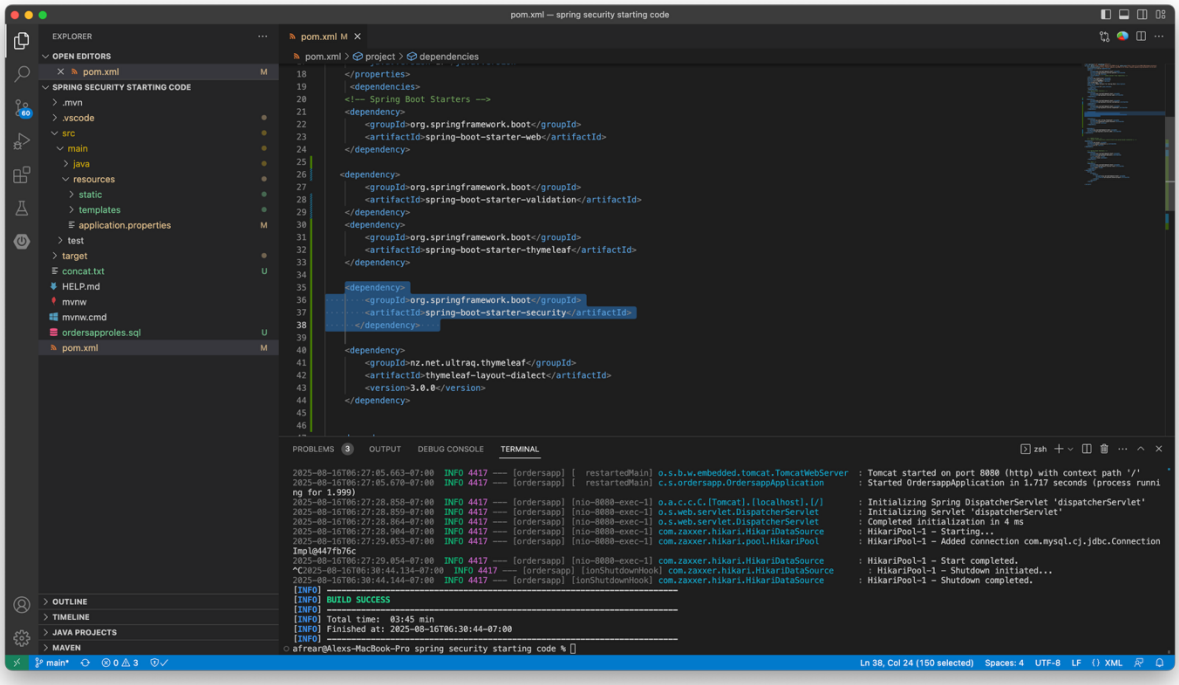

Figure 3 `spring-boot-starter-security` added to `pom.xml`, enabling Spring Security for the application.

Default Spring Security Login Page

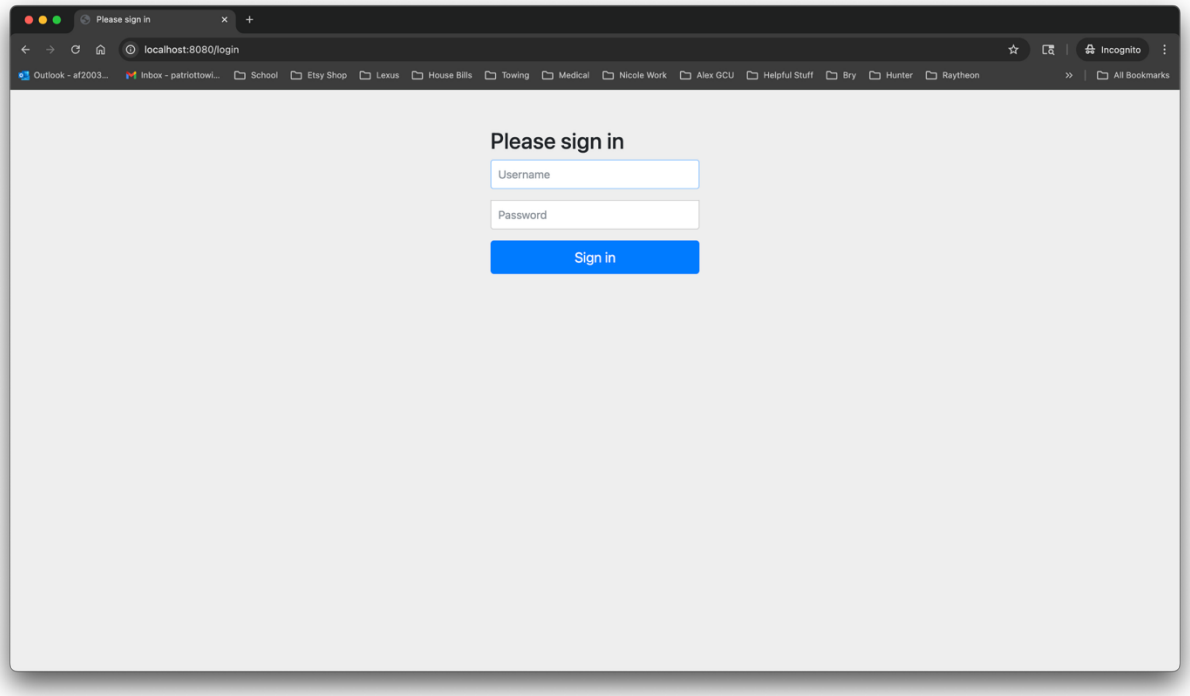

Figure 4 Default login page appears after adding the security starter, confirming Spring Security is active.

1.3 Create the Security Configuration Class

Security Package Created

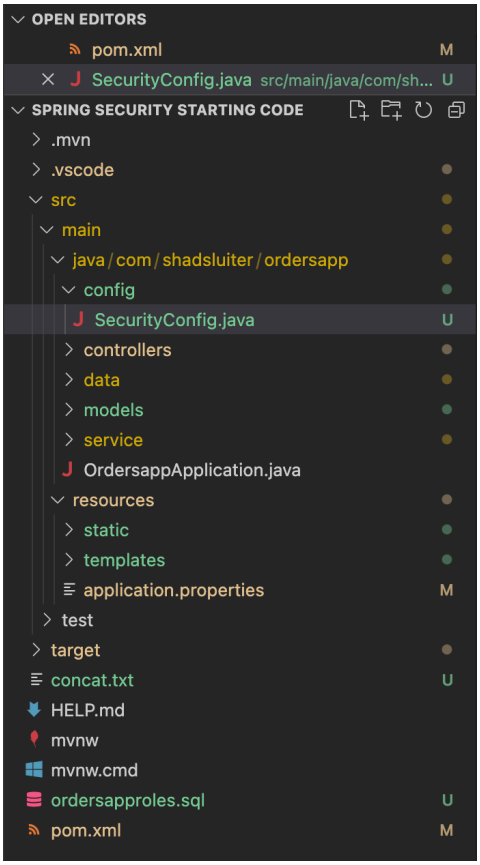

Figure 5 `config/` package created to hold Spring Security configuration classes.

SecurityConfig Class Implemented

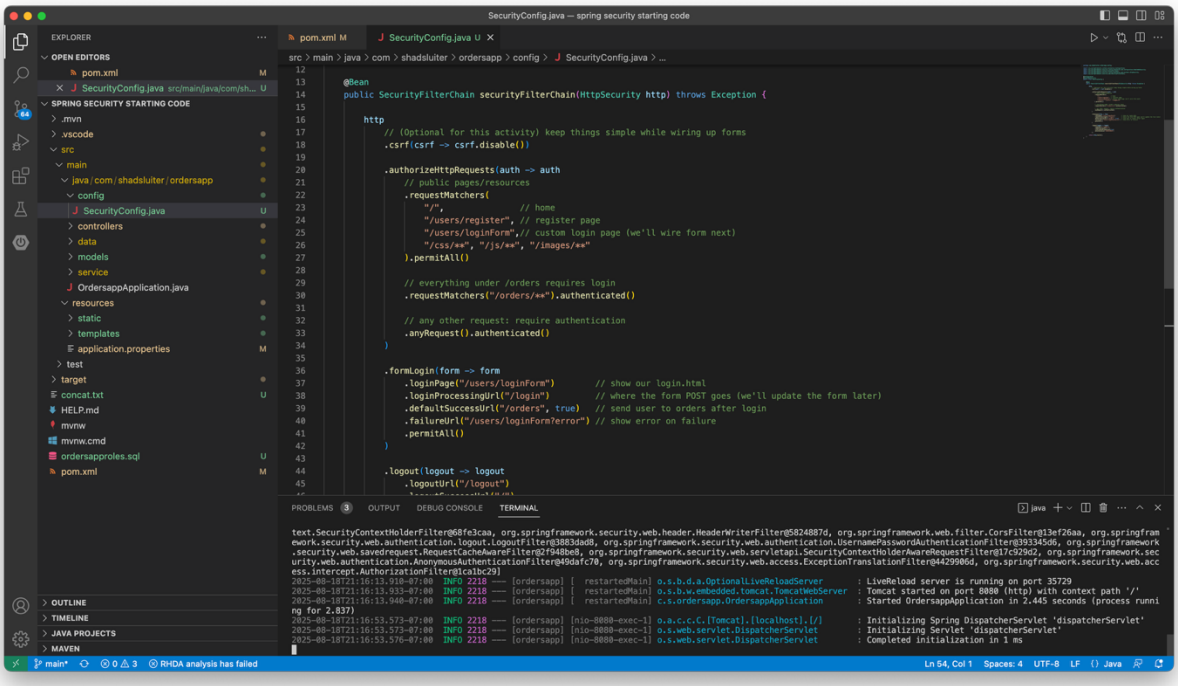

Figure 6 `SecurityConfig` defines the `SecurityFilterChain` with public routes, authenticated `/orders/**`, custom login page, and logout behavior.

1.4 Password Encoding Configuration

Password Encoder Bean (BCrypt)

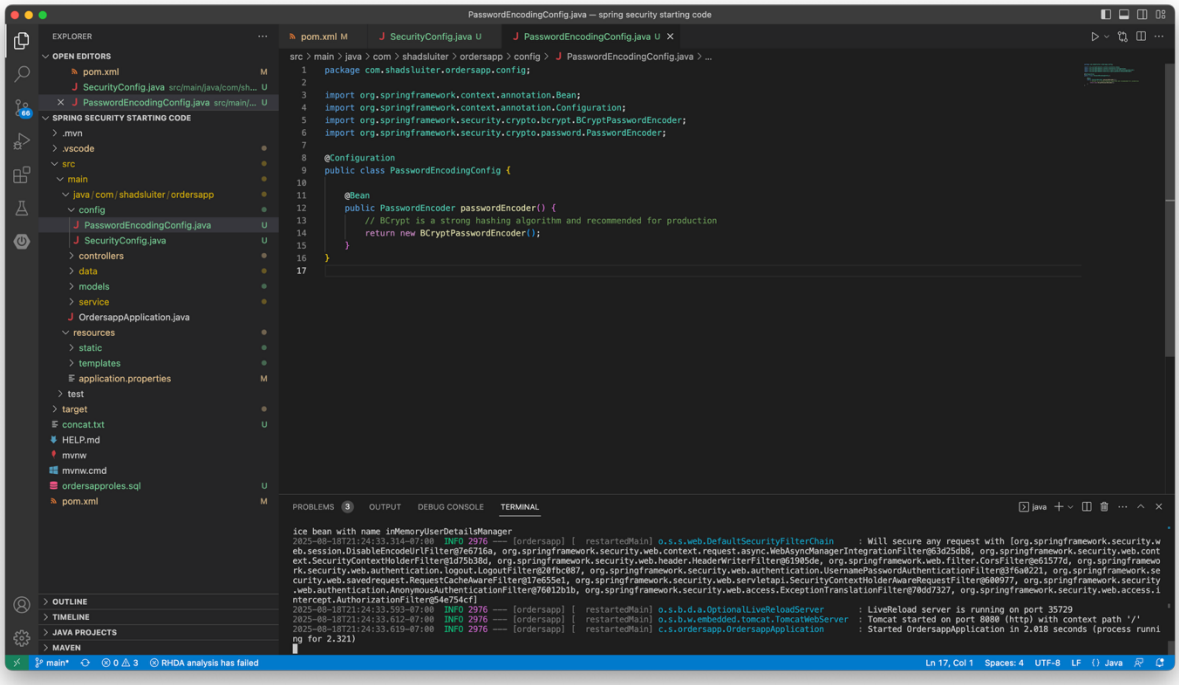

Figure 7 BCrypt `PasswordEncoder` bean defined; recommended for securely hashing user passwords.

1.5 Update the User Service for Authentication

UserDetailsService Implemented (DB-backed)

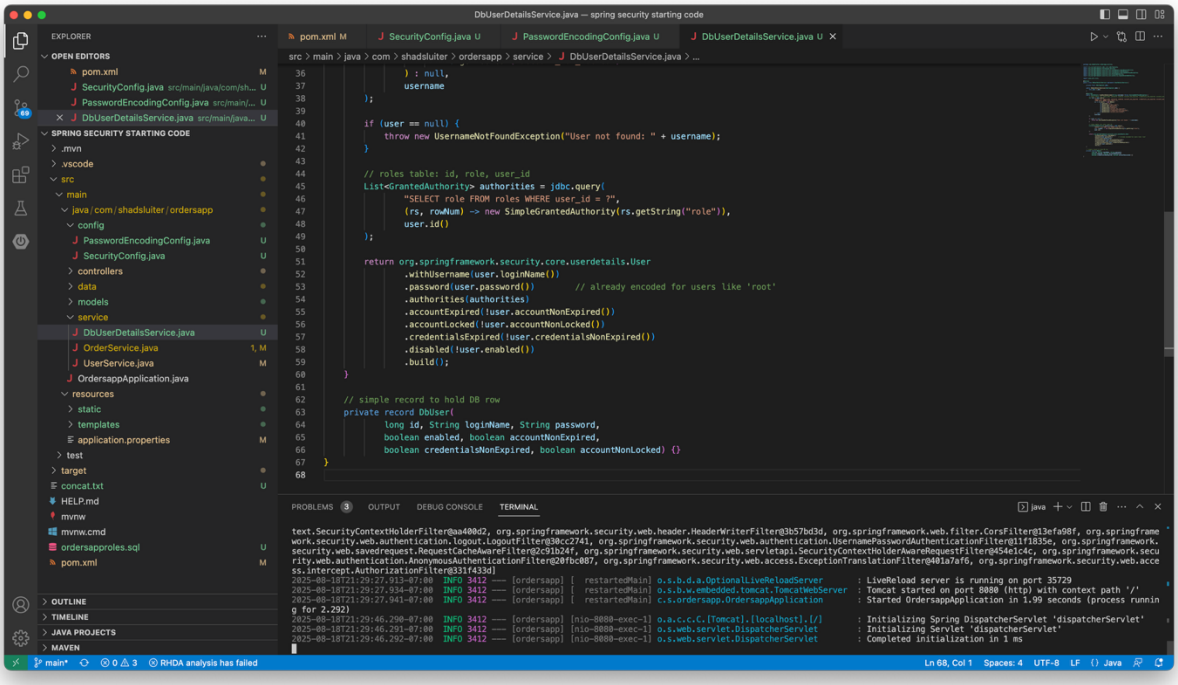

Figure 8 `DbUserDetailsService` loads users from `users` and roles from `roles`, returning a Spring Security `User` with authorities.

Part 2 – Custom Login Page

2.1 – Custom Login HTML

Custom Login HTML

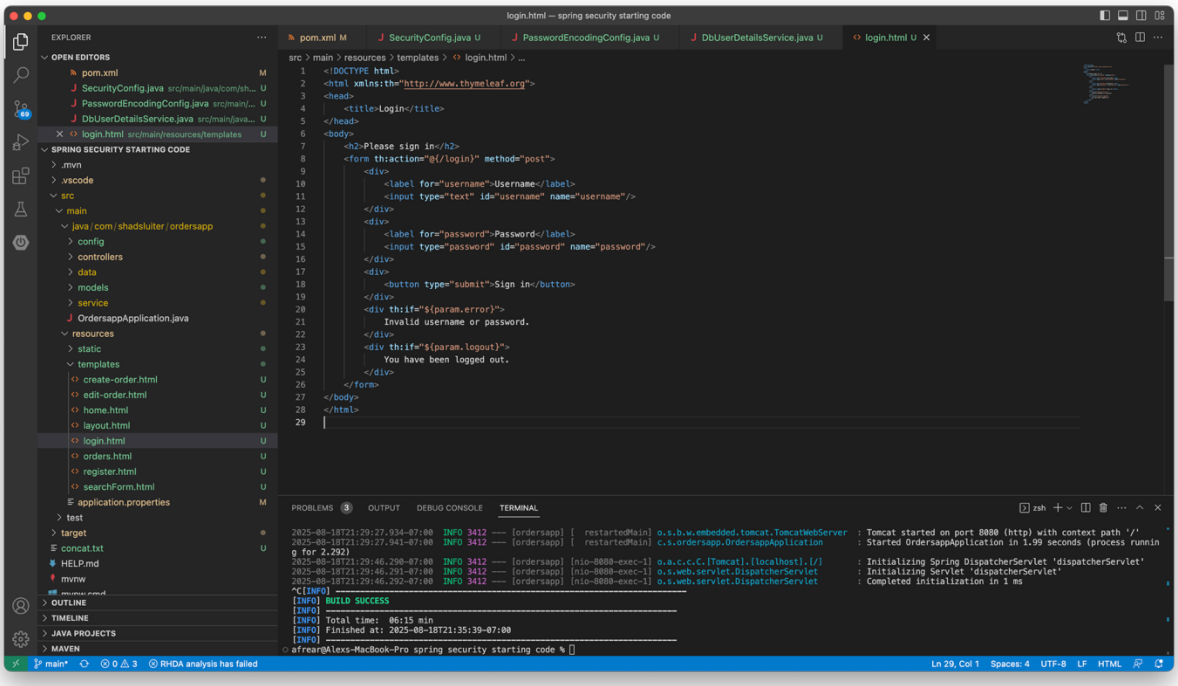

Figure 9 Custom `login.html` created in the templates folder, configured to post to `/login` and display error/logout messages.

2.2 – Update SecurityConfig to Use Custom Login

SecurityConfig Custom Login Update

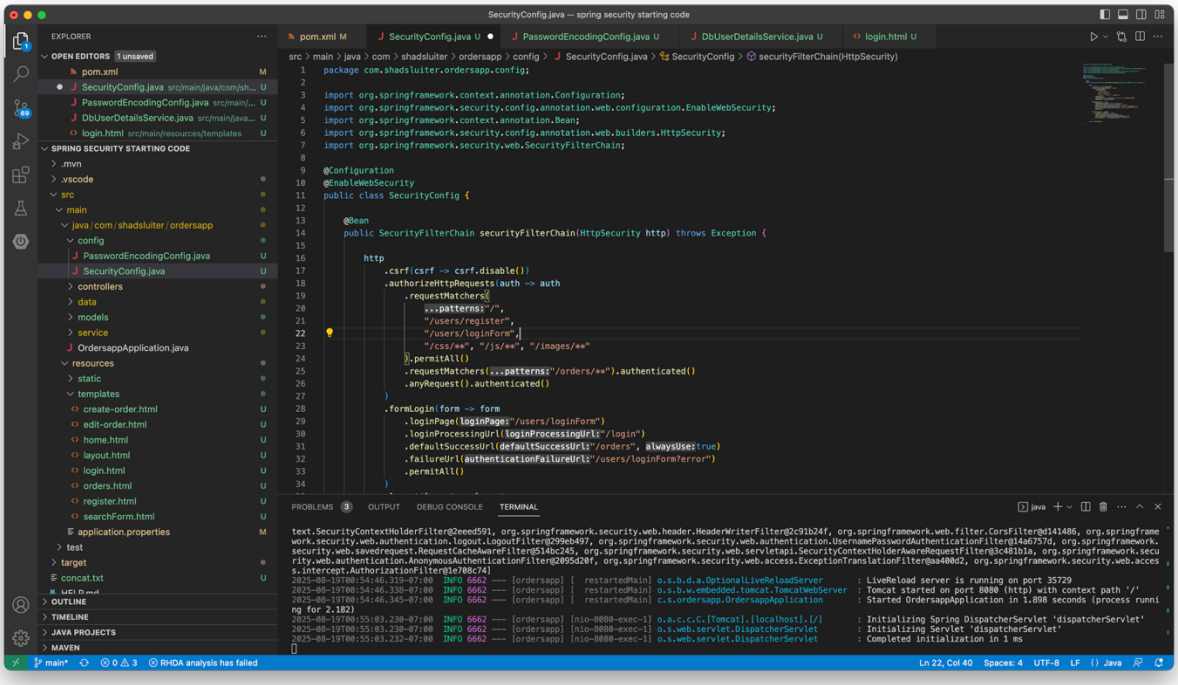

Figure 10 `SecurityConfig` updated to permit `/login` and to use `.loginPage("/login")` with `.failureUrl("/login?error")`

2.3 – Test the Custom Login Page

Login Page (Custom) Redirect

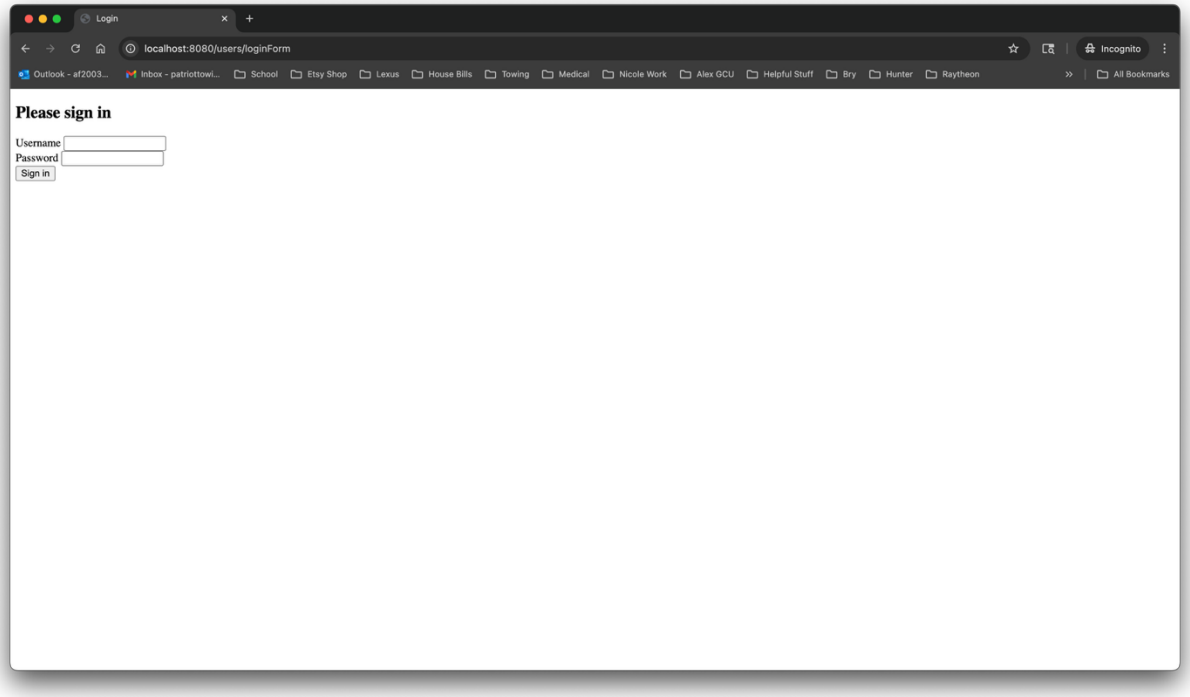

Figure 11 Navigating to `/orders` redirects to the custom `/login` page served from `templates/login.html`.

2.4 – Verify Access After Login

Orders Page After Login

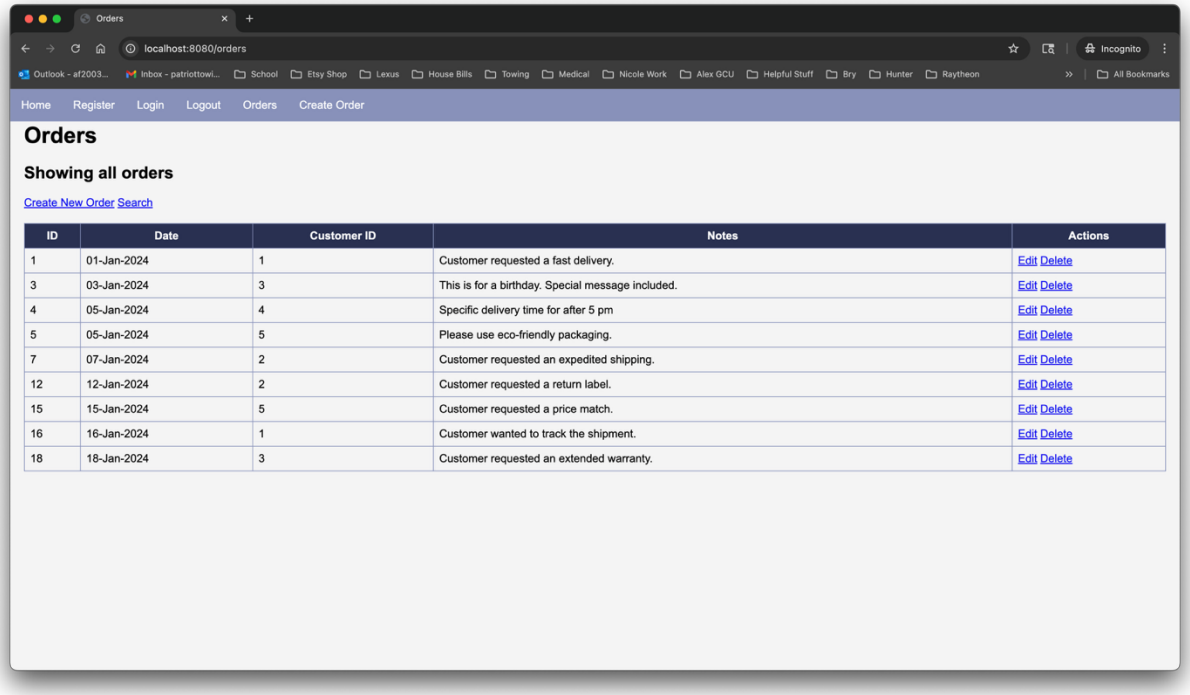

Figure 12 After registering with BCrypt-encoded password and signing in, the protected `/orders` page is accessible.

Logout Implemented as POST

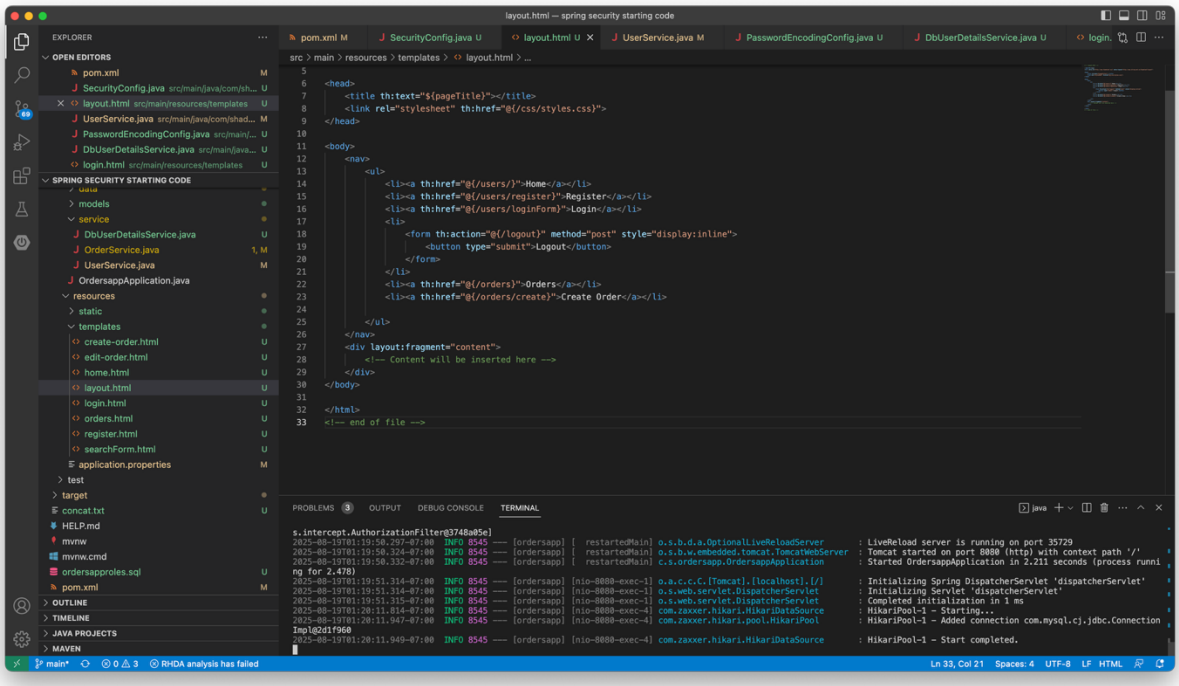

Figure 13 Navbar updated to submit a POST to `/logout`, matching Spring Security's expected logout method.

Logout and Redirect to Home

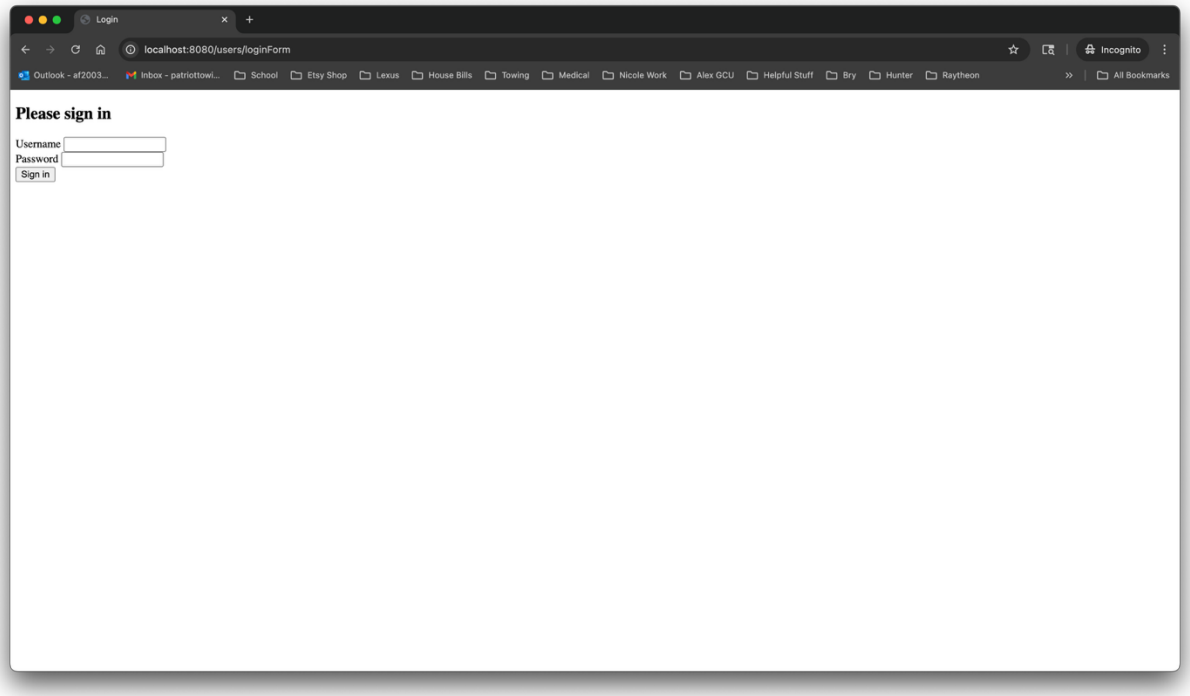

Figure 14 After clicking Logout, the session is invalidated and the user is redirected; accessing `/orders` now prompts for login.

Part 3: Role-Based Access Control (RBAC)

3.1 – Add Roles to the User Entity

RBAC Rules in SecurityConfig

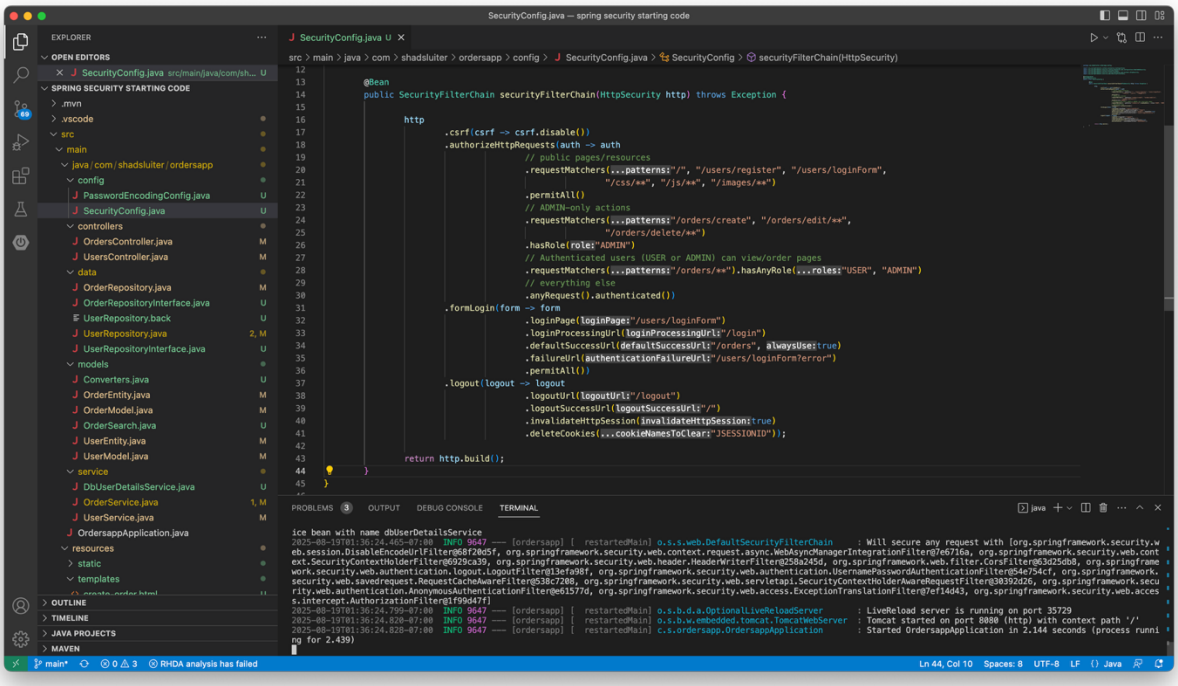

Figure 15 Security rules restrict `/orders/create|edit|delete` to `ROLE_ADMIN`, while `/orders/**` is available to authenticated users.

3.2 – Verify USER is denied on admin routes

USER Blocked from Admin Route

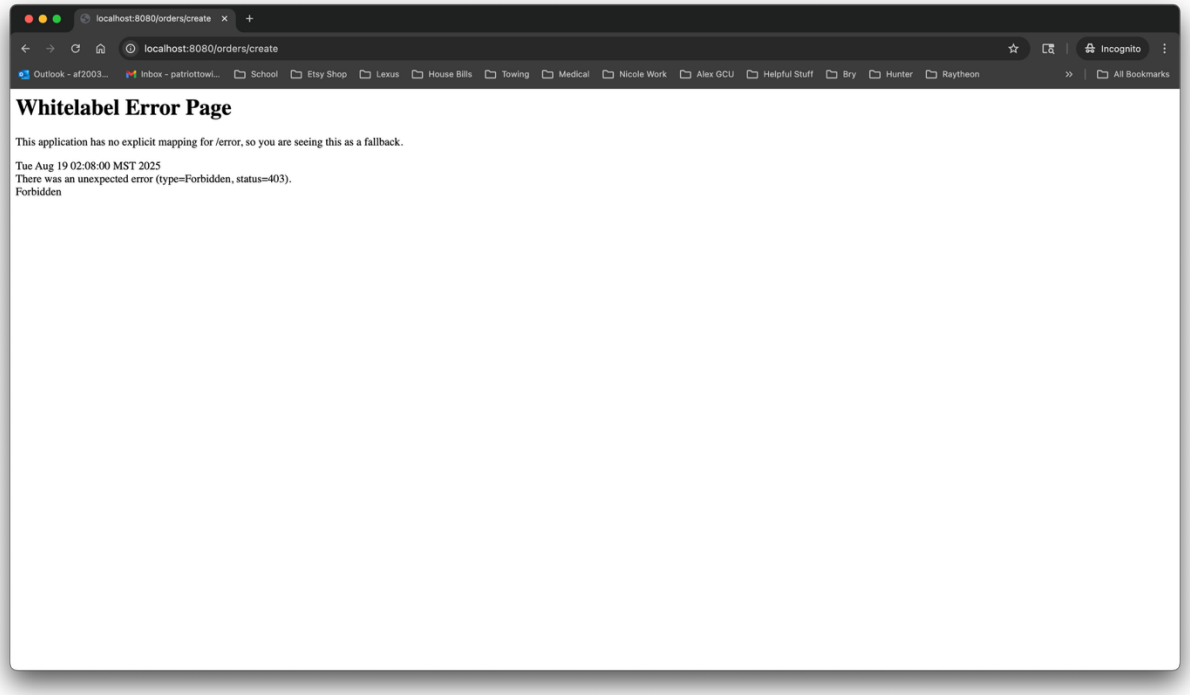

Figure 16 A `ROLE_USER` account attempts to open `/orders/create` and receives 403 Forbidden per RBAC rules.

3.3 – Verify ADMIN can access admin routes

Admin Can Access Create Order

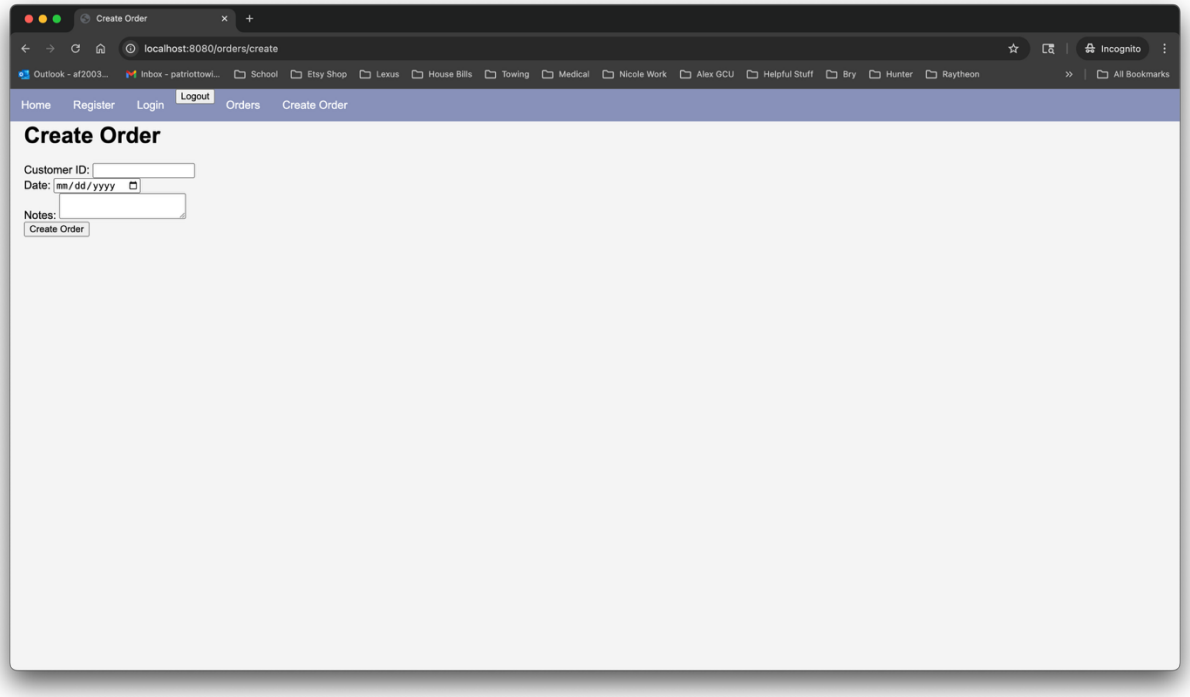

Figure 17 User with `ROLE_ADMIN` successfully accesses the `/orders/create` page.

3.4 – Show/Hide Navbar Items Based on Login & Role

Thymeleaf Security Extras Dependency

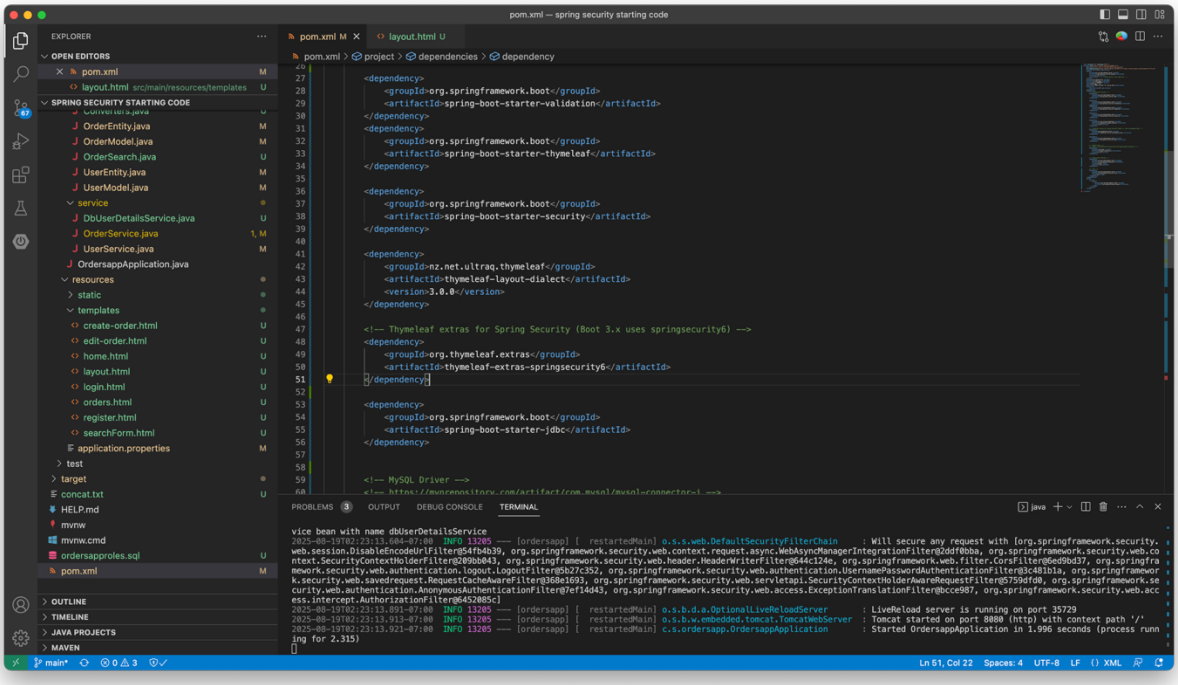

Figure 18 `pom.xml` updated with `thymeleaf-extras-springsecurity6` to enable `sec:*` attributes.

Navbar (Logged Out)

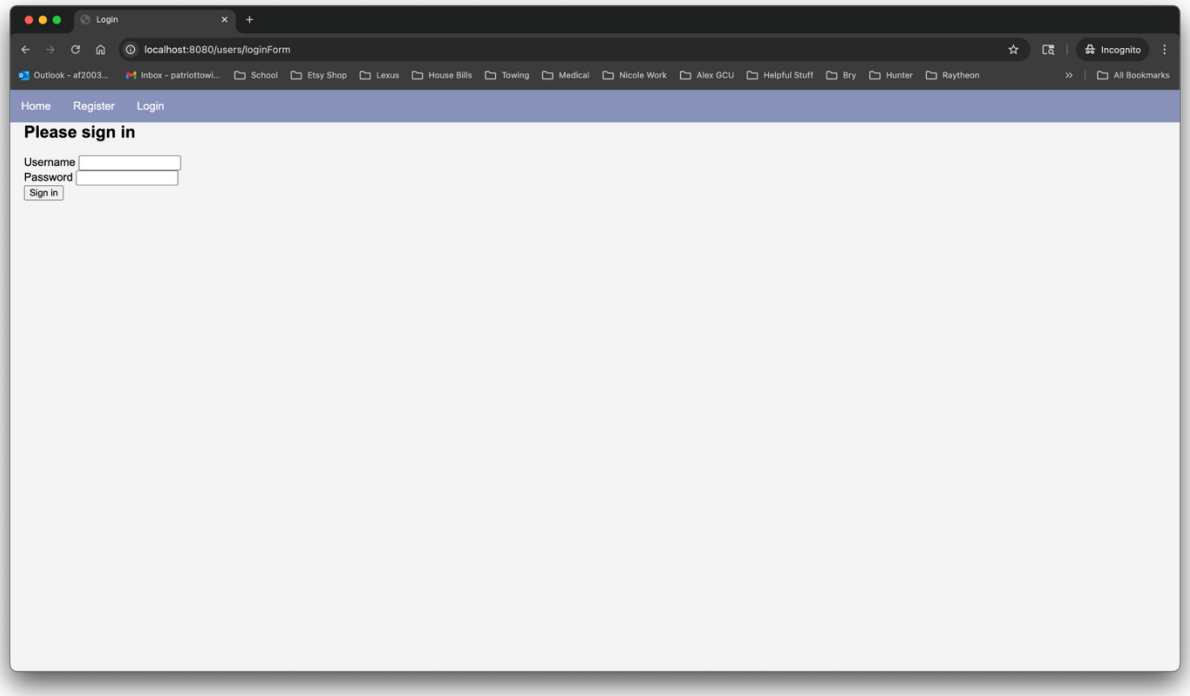

Figure 19 When not authenticated, only Home, Register, and Login are visible.

Navbar (Admin — Create Visible)

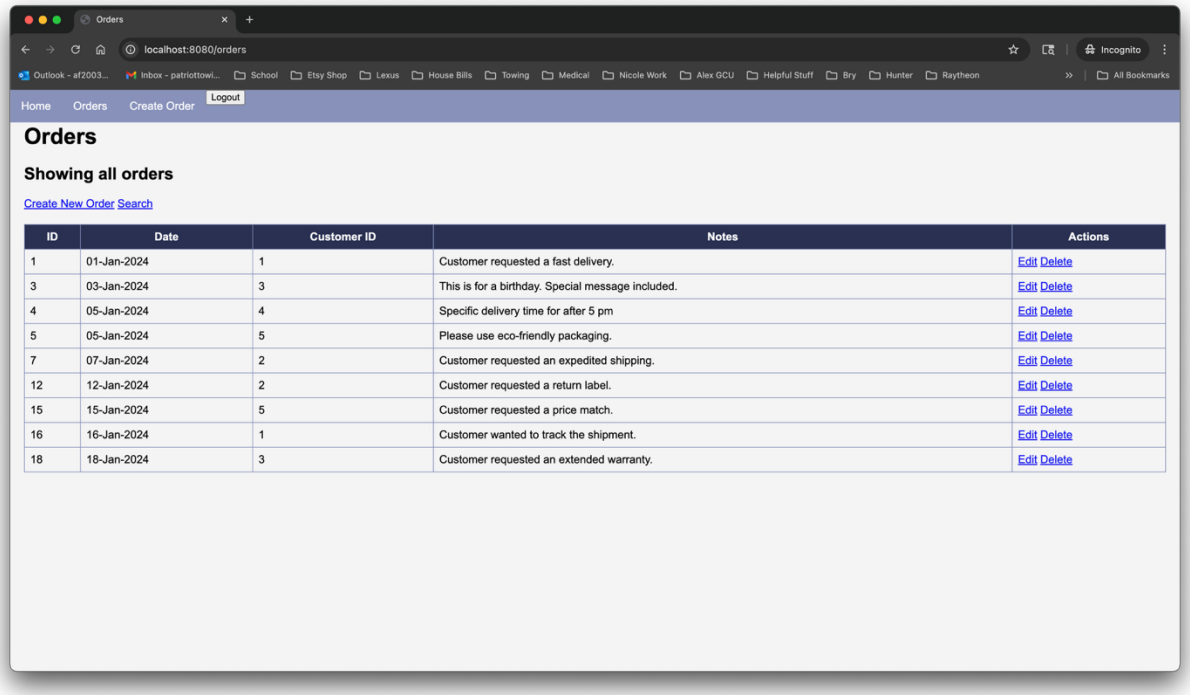

Figure 20 When logged in as `ROLE_ADMIN`, Create Order is visible along with Orders and Logout.

3.5 – Add Method-Level Security (defense in depth)

Enable Method-Level Security

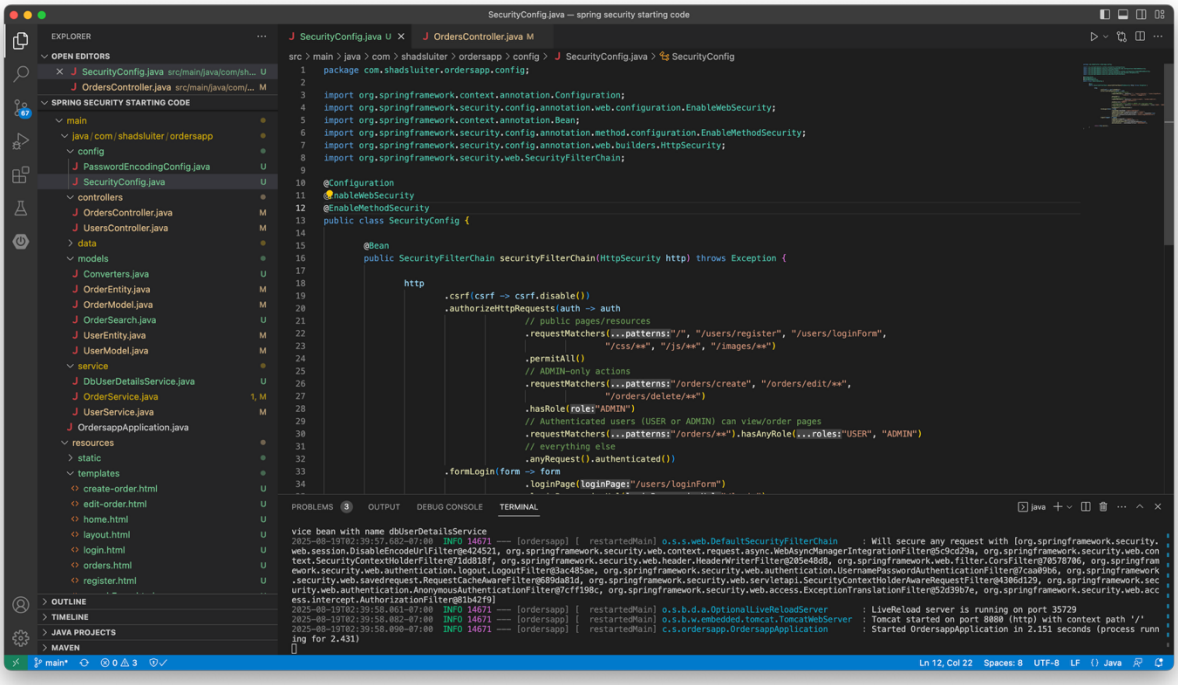

Figure 21 `SecurityConfig` updated with `@EnableMethodSecurity` to allow `@PreAuthorize` checks.

PreAuthorize on OrdersController (Admin Only)

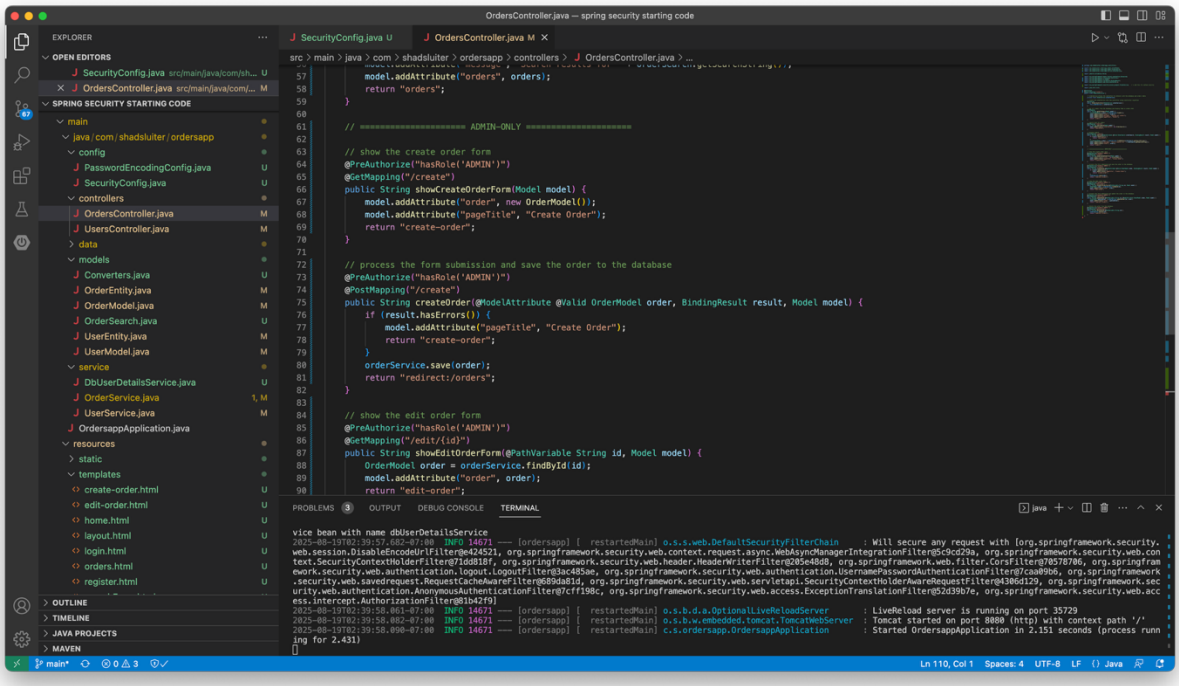

Figure 22 `@PreAuthorize('hasRole('ADMIN'))` applied to create/edit/delete endpoints in `OrdersController`.

3.6 – Hide admin actions in the UI (Edit/Delete/Create link)

Orders Page Hides Admin Actions for USER

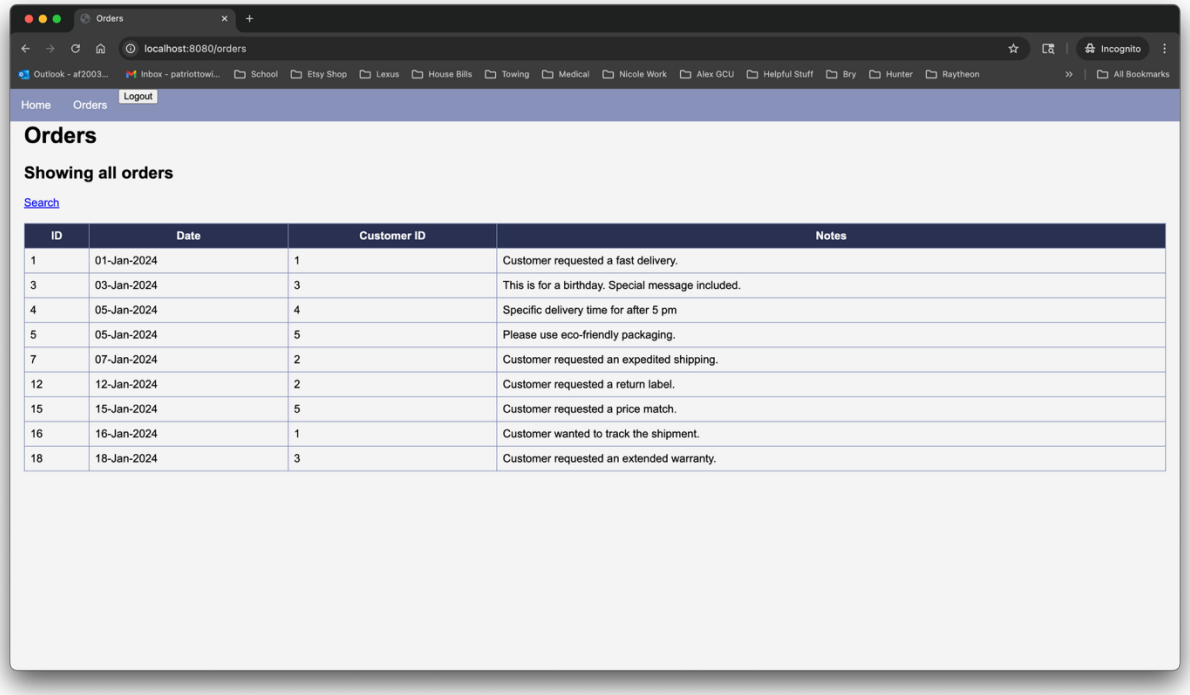

Figure 23 Logged in as `ROLE_USER`; the Orders page hides Create link and the Actions column.

Orders Page Shows Admin Actions

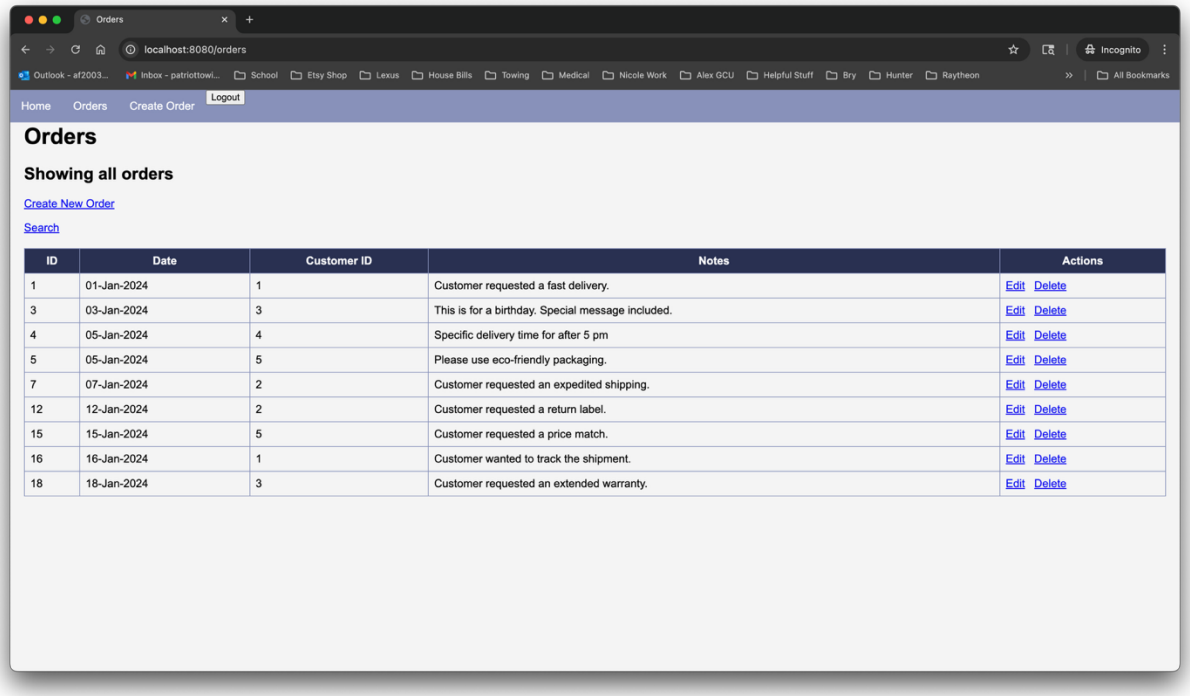

Figure 24 Logged in as `ROLE_ADMIN`; the Orders page shows Create, Edit, and Delete options.

3.7 – Add a Custom “Access Denied” (403) Page

Custom Access Denied Page

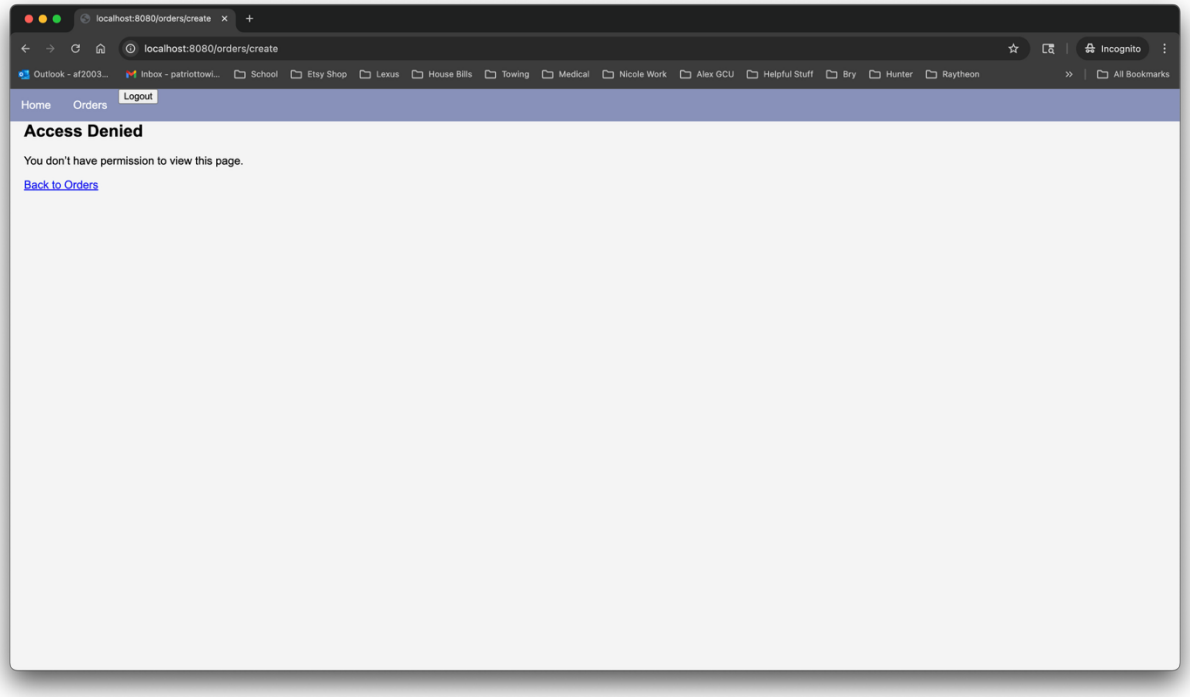

Figure 25 `/orders/create` denied to `ROLE_USER` shows the custom 403 page via `exceptionHandling().accessDeniedPage('/access-denied')`.

Part 4 – Summary of Key Concepts

When I was working through this activity, I realized that Spring Security relies on a few key classes to handle how authentication and authorization work. For example, the `SecurityFilterChain` is where the rules are defined for which endpoints need authentication and what roles are allowed to access them. The `AuthenticationManager` is the piece that processes login attempts and checks if the credentials are valid, while the `UserDetailsService` is responsible for pulling in user data like usernames, passwords, and roles from either memory or a database. I also learned how important the `PasswordEncoder` is because it ensures that passwords are stored securely as hashes instead of plain text. All of this gets tied together with the `HttpSecurity` class, which gives us a way to configure web-based security in a clean, structured way.

Another big concept that stood out to me is how the filter chain works in Spring Security. Every request that comes into the application goes through this chain before it ever reaches a controller. Each filter has its own job, like checking if a user is authenticated, making sure the user has the right authorization, or applying protections like CSRF. If a request doesn't meet the requirements, it's blocked right there with either a 401 Unauthorized or a 403 Forbidden, depending on the issue. Only if the request passes all the filters does it actually move on to the controller. I think this setup makes sense because it keeps the controller focused on business logic, while the filter chain handles all the security in one place.